Redis 面试八股笔记

面向 Go 后端 / AI Agent 开发实习

Redis 面试八股笔记

一、基础与数据结构

1. Redis 高性能原因与线程模型
2. 网络模型与后台线程
3. 数据类型与底层编码
4. SDS 简单动态字符串
5. 跳表（ZSet 底层）
6. 哈希表 dict 与渐进式 rehash
7. 压缩列表 ziplist 与 listpack

二、持久化

8. RDB 快照
9. AOF 日志与重写
10. 混合持久化与选型

三、过期删除与内存淘汰

11. 过期删除策略
12. 内存淘汰策略（LRU / LFU）

四、高可用：主从、哨兵、集群

13. 主从复制：全量与增量
14. 哨兵 Sentinel 与故障转移
15. Redis Cluster 与哈希槽
16. 脑裂与数据丢失

五、缓存问题

17. 缓存穿透与布隆过滤器
18. 缓存击穿
19. 缓存雪崩
20. 大 key 与热 key
21. KEYS 与 SCAN
22. Redis 变慢排查思路

六、分布式锁

23. Redis 分布式锁
24. Redis / ZooKeeper / etcd 锁对比

七、缓存一致性

25. Cache Aside 与缓存一致性
26. 其他缓存读写模式

八、其他常考
27. 事务、Lua 脚本与 Pipeline
28. 典型应用：延迟队列、排行榜、限流
29. 秒杀场景与防超卖

> 使用说明：按「主题 → 知识点」组织的结构化笔记，主体参考小林coding《图解Redis》及其面试题合集的知识框架，已重新整理改写。先记加粗关键词，再展开。涉及版本差异处已标注版本号。MySQL 见 MySQL 面试八股笔记。Go 秒杀综合实战见 Go 秒杀系统，其中【R-N】指向本文知识点编号。

Redis 面试八股笔记

一、基础与数据结构

1. Redis 高性能原因与线程模型

- **纯内存操作**：读写都在内存，避免磁盘 IO。
- **单线程执行命令**：避免多线程上下文切换和锁竞争，天然保证单条命令原子性。
- **IO 多路复用**：基于 epoll/kqueue 的 Reactor 模型，一个线程处理大量连接。
- **高效数据结构**：SDS、listpack、quicklist、跳表、渐进式 rehash 的 dict 等。
- 「单线程」指**命令执行（网络读写+命令处理）主流程是单线程**；持久化 fork 子进程、`UNLINK` / `lazyfree` 异步释放内存等由后台线程完成。
- **Redis 6.0 引入多线程 IO**（`io-threads`，默认关闭）：多线程只负责网络读写和协议解析，**命令执行仍是单线程**。

延伸 / 易错点

- **瓶颈**：通常是**内存和网络带宽**而非 CPU；大 key、`KEYS *`、高复杂度命令会阻塞主线程。
- **6.0 加多线程 IO 的原因**：高并发下网络读写占主线程时间多，多线程 IO 提升吞吐，命令执行仍无锁。

2. 网络模型与后台线程

- **单 Reactor 单线程（6.0 前）**：`epoll_create` 创建 epoll，监听 socket 注册连接事件 → 事件循环中 `epoll_wait` 等待事件：
  - 连接事件 → accept，为客户端 socket 注册**读事件**；
  - 读事件 → 读取请求、解析并执行命令，结果写入客户端发送缓冲区，客户端加入**待写队列**；
  - 每轮进入 `epoll_wait` 前（beforeSleep）先把待写队列的数据发出，发不完才注册**写事件**下轮继续发。

- **6.0 多 IO 线程**：主线程把读写 socket 的工作分发给 IO 线程并行处理，**命令仍由主线程串行执行**。
- **后台线程（BIO）**：关闭文件、AOF 刷盘（fsync）、lazyfree 异步释放内存（`UNLINK`、`FLUSHALL ASYNC`）三类耗时任务交给后台线程，避免阻塞主线程。

3. 数据类型与底层编码

类型	底层编码（以 Redis 7.x 为准）	典型场景
String	`int` / `embstr`（≤44 字节）/ `raw`，底层 SDS	缓存、计数器、分布式锁、Session
List	`quicklist`（双向链表，节点是 listpack；3.2~6.x 节点是 ziplist）	消息队列、最新列表
Hash	`listpack`（元素少且小，7.0 前为 ziplist）/ `hashtable`	对象存储、购物车
Set	`intset`（全整数且少）/ `listpack`（7.2+）/ `hashtable`	去重、共同好友、抽奖
ZSet	`listpack`（元素少，7.0 前为 ziplist）/ `skiplist + dict`	排行榜、延迟队列

- 扩展类型：**Bitmap**（签到、在线状态）、**HyperLogLog**（UV 基数统计，约 12KB，标准误差 0.81%）、**GEO**（附近的人，底层 ZSet + GeoHash）、**Stream**（5.0，消息队列，支持消费组和 ACK）。
- 编码转换阈值可配置，如 `hash-max-listpack-entries 128`、`hash-max-listpack-value 64`，超过即转为 hashtable，**只升级不降级**。

延伸 / 易错点

- **ZSet 同时用跳表 + dict**：dict 实现 **O(1) 按成员查分数**，跳表实现**按分数有序和范围查询**。
- **embstr vs raw**：embstr 将 redisObject 和 SDS **一次分配、内存连续**，只读；修改后转为 raw（两次分配）。

4. SDS 简单动态字符串

- **O(1) 获取长度**：记录 `len`。
- **二进制安全**：以 len 判断结束而非 `\0`，可存图片等任意数据。
- **杜绝缓冲区溢出**：修改前检查 `alloc` 空间，不够自动扩容。
- **减少内存重分配**：**空间预分配**（修改后长度 < 1MB 时分配同等大小的空闲空间，即翻倍；≥ 1MB 时额外多分配 1MB）+ **惰性空间释放**。
- 结尾仍保留 `\0`，可兼容部分 C 字符串函数。
- 3.2 后按长度使用 sdshdr8/16/32/64 不同头部，节省内存。

延伸 / 易错点

- **容量上限**：单个 String value 最大 **512MB**。

5. 跳表（ZSet 底层）

- 跳表 = **多层有序链表**，上层是下层的「索引」，查找从最高层开始逐层下降，平均 **O(log N)**。
- 新节点层数**随机生成**：每升一层概率 **p = 1/4**，最大层数 32（`ZSKIPLIST_MAXLEVEL`）。
- Redis 跳表节点额外有**后退指针**（支持逆序遍历）和每层的 **span 跨度**（用于 O(log N) 计算排名 `ZRANK`）。
- 选择跳表的原因：
  - **范围查询简单**：找到起点后沿底层链表遍历即可；红黑树需要中序遍历。
  - **实现简单、易调试**，插入删除只需改局部指针，无需旋转再平衡。
  - **内存灵活**：通过调整 p 平衡内存与性能。
  - 不用 B+ 树：B+ 树为**减少磁盘 IO** 设计，Redis 在内存中，没有这种需求。

延伸 / 易错点

- **插入流程**：自顶向下找每层前驱（记录 update[]）→ 随机层数 → 逐层插入并更新 span。

6. 哈希表 dict 与渐进式 rehash

- dict 内部有**两个哈希表 ht[0]、ht[1]**，链地址法解决冲突。
- 扩容触发（负载因子 = 元素数 / 桶数）：
  - 负载因子 **≥ 1** 且当前**没有执行 BGSAVE/BGREWRITEAOF**（避免 fork 期间写时复制放大内存）；
  - 负载因子 **≥ 5** 时强制扩容。
  - 负载因子 < 0.1 时缩容。
- **渐进式 rehash**：为 ht[1] 分配空间，`rehashidx` 从 0 开始；之后**每次增删改查顺带迁移一个桶**，另外定时任务（serverCron）也会分批迁移；完成后 ht[1] 变为 ht[0]。
- rehash 期间：**查找/删除/更新先查 ht[0] 再查 ht[1]，新增只写 ht[1]**。

延伸 / 易错点

- **渐进式的意义**：一次性迁移百万级 key 会长时间阻塞单线程主流程。

7. 压缩列表 ziplist 与 listpack

- ziplist 是连续内存的紧凑列表，每个 entry 记录**前一个节点长度 prevlen**（前节点 < 254 字节用 1 字节，否则 5 字节）。
- **连锁更新**：插入一个较大节点，后一节点 prevlen 从 1 字节变 5 字节，长度增加又可能导致再后一个节点也需扩展，最坏 $O(N^2)$。
- **listpack（5.0 引入，7.0 全面替代 ziplist）**：每个 entry **只记录自身长度**（encoding + data + backlen），不依赖前节点长度，**彻底消除连锁更新**；仍支持从后向前遍历。

延伸 / 易错点

- **小数据用紧凑结构的原因**：内存连续、无指针开销、CPU 缓存友好；元素少时 O(N) 遍历也很快。

二、持久化

8. RDB 快照

- RDB = 某一时刻内存数据的**二进制快照**。
- 触发：`SAVE`（主线程执行，阻塞，不推荐）、`BGSAVE`（**fork 子进程**生成）、配置 `save 900 1` 等自动触发、主从全量复制、`SHUTDOWN` 时。
- **写时复制（COW）**：fork 后父子进程共享物理内存页，父进程修改某页时才复制该页，子进程看到的是 fork 时刻的一致性快照。
- 优点：**文件紧凑、恢复速度快**，适合备份和全量复制。
- 缺点：**两次快照间的数据会丢失**；fork 本身会阻塞主线程（与页表大小相关），大内存实例 fork 耗时明显；写多时 COW 导致内存最多接近翻倍。

延伸 / 易错点

- **fork 阻塞点**：复制**页表**等进程数据结构，而非全部数据。

9. AOF 日志与重写

- AOF 记录**每条写命令**（文本协议），重启时重放恢复；**先执行命令再写日志**（不阻塞当前命令、无需语法检查，但可能丢失刚执行的命令）。
- 写回策略 `appendfsync`：
  - `always`：每条命令 fsync，最安全最慢；
  - `everysec`（默认）：每秒 fsync，**最多丢约 1 秒数据**；
  - `no`：由操作系统决定刷盘时机。
- **AOF 重写（BGREWRITEAOF）**：fork 子进程**根据当前内存数据**生成最简命令集（而非分析旧 AOF），压缩体积。重写期间新命令同时写入 AOF 缓冲和 **AOF 重写缓冲区**，子进程完成后追加并原子替换。
- **Redis 7.0 Multi-Part AOF**：AOF 拆为 base 文件 + 多个 incr 增量文件 + manifest 清单，重写期间新命令直接写新 incr 文件，**不再需要重写缓冲区**。

延伸 / 易错点

- **用子进程而非线程重写**：子进程有独立内存快照（COW），无需加锁即可保证数据一致。

10. 混合持久化与选型

- **混合持久化（4.0 引入，`aof-use-rdb-preamble yes`，5.0 起默认开启）**：AOF 重写时，前半部分写 **RDB 格式全量数据**，后半部分为重写期间的**增量 AOF 命令**。兼顾恢复速度和数据完整性。
- 选择：

- 能容忍分钟级丢失、追求恢复快 → RDB；
- 数据安全要求高 → AOF everysec；
- 生产推荐 **RDB + AOF（混合持久化）**；纯缓存场景可都关闭。

- 重启加载优先级：**开启 AOF 时优先加载 AOF**（更完整）。

延伸 / 易错点

- **混合持久化缺点**：AOF 文件包含 RDB 二进制部分，可读性差，旧版本无法识别。

三、过期删除与内存淘汰

11. 过期删除策略

- 过期时间保存在**过期字典 expires**（key → 过期时间戳）。
- **惰性删除**：访问 key 时检查是否过期，过期则删除。CPU 友好，但不访问的过期 key 会占内存。
- **定期删除**：serverCron 周期执行（`hz` 默认 10，即每秒 10 次），每次**随机抽取一批设置了过期时间的 key**（经典描述为 20 个）删除其中已过期的；若过期比例超过阈值（经典描述 25%）则继续抽样，并有**执行时长上限**防止阻塞。新版本可通过 `active-expire-effort` 调整力度。
- Redis 采用 **惰性删除 + 定期删除** 组合。

延伸 / 易错点

- **不用定时删除的原因**：每个 key 一个定时器，大量 key 时 CPU 开销过大。
- **主从下的过期 key**：**从库不主动删除过期 key**，由主库删除后向从库同步 `DEL`；3.2 起从库读取已过期 key 返回空。

12. 内存淘汰策略（LRU / LFU）

- 达到 `maxmemory` 后触发，共 **8 种**：
  - `noeviction`（**默认**）：不淘汰，写命令报错；
  - 所有 key 中：`allkeys-lru`、`allkeys-lfu`、`allkeys-random`；
  - 设置了过期时间的 key 中：`volatile-lru`、`volatile-lfu`、`volatile-random`、`volatile-ttl`（优先淘汰剩余寿命最短的）。
- **近似 LRU**：redisObject 中 24 bit 记录最近访问时间；淘汰时**随机采样 `maxmemory-samples`（默认 5）个 key**，淘汰最久未访问的（3.0 起有候选淘汰池优化）。不维护全局链表，节省内存和开销。
- **LFU（4.0 引入）**：同样 24 bit 拆分为 **16 bit 最近递减时间 + 8 bit 对数访问计数器**（最大 255，概率递增）；计数会随时间衰减（`lfu-decay-time`），解决 LRU 「偶尔访问一次就被当成热点」的问题。

延伸 / 易错点

- **选型**：缓存场景通常 `allkeys-lru`；热点明显、访问频率差异大时选 `allkeys-lfu`。

四、高可用：主从、哨兵、集群

13. 主从复制：全量与增量

- **全量复制**：
  1. 从库发送 `PSYNC ? -1`（首次，不知道主库 replid）；
  2. 主库回复 `FULLRESYNC replid offset`，执行 **BGSAVE** 生成 RDB 发送给从库；
  3. 期间新写命令写入该从库的 **replication buffer**；
  4. 从库清空旧数据、加载 RDB，主库再发送缓冲中的命令。
- **命令传播**：之后主库将写命令持续异步发送给从库，基于**长连接**。
- **增量复制（2.8 起 PSYNC）**：断线重连后从库发送 `PSYNC replid offset`；主库在**复制积压缓冲区 repl_backlog_buffer（环形缓冲区）** 中查找，offset 仍在缓冲区内则回复 `CONTINUE` 只发送缺失部分，否则退化为全量复制。
- Redis 复制默认**异步**，主库不等待从库确认。

延伸 / 易错点

- **避免频繁全量复制**：调大 `repl-backlog-size`（建议 ≥ 写入速率 × 平均断线时长 × 2）。
- **主从级联**：「主-从-从」结构减轻主库 fork 和网络压力。

14. 哨兵 Sentinel 与故障转移

- 作用：**监控、自动故障转移、通知、配置提供**（客户端通过哨兵获取主库地址）。
- **主观下线（SDOWN）**：单个哨兵 PING 超过 `down-after-milliseconds` 未收到有效回复。
- **客观下线（ODOWN）**：询问其他哨兵，认为下线的数量 ≥ **quorum** 时判定（只针对主库）。
- **选举 Leader 哨兵**：类 Raft，获得**半数以上**哨兵投票且票数 ≥ quorum 才能成为 Leader，执行故障转移。因此哨兵至少部署 **3 个**。
- **选新主库**：过滤掉下线/网络不佳的从库 → `replica-priority` 小者优先 → **复制偏移量最大**者优先 → **runid 最小**者。
- 执行：新主 `REPLICAOF NO ONE`，其余从库指向新主，旧主恢复后变为从库，通过发布订阅通知客户端。

延伸 / 易错点

- **哨兵互相发现**：通过主库的 `__sentinel__:hello` 频道发布订阅；通过 `INFO` 命令从主库获取从库列表。

15. Redis Cluster 与哈希槽

- **去中心化**分片集群，数据分到 **16384 个哈希槽**：`slot = CRC16(key) mod 16384`，每个主节点负责一部分槽，每个主节点可有从节点。
- 节点之间通过 **Gossip 协议**交换状态（PING/PONG/MEET/FAIL）。
- **客户端重定向**：
  - `MOVED`：槽已**永久**迁移到其他节点，客户端更新本地槽映射；
  - `ASK`：槽**迁移中**，仅本次请求转向目标节点（先发 `ASKING`）。
- **Hash Tag**：`{user1}:a` 与 `{user1}:b` 只按 `{}` 内计算 slot，保证落在同一节点，用于多 key 操作/Lua 脚本。
- 故障转移：节点间互相检测 PFAIL → 多数主节点确认 FAIL → 从节点发起选举，**获得多数主节点投票**后升级。
- 为什么 16384：心跳包携带槽位图 **16384 bit = 2KB**，65536 则需 8KB，带宽开销大；作者建议集群主节点**不超过 1000 个**，16384 槽足够。

延伸 / 易错点

- **集群限制**：跨 slot 的多 key 操作（MGET、事务、Lua）不支持（报 CROSSSLOT）；只能使用 db0。
- **哈希槽 vs 一致性哈希**：槽是固定逻辑分区，迁移以槽为单位，扩缩容和数据分布更可控。

16. 脑裂与数据丢失

- 主库因网络分区与哨兵/从库失联但仍在运行，哨兵选出新主后出现**两个主库**；旧主恢复后降为从库并**全量同步新主数据，期间写入旧主的数据丢失**。
- 缓解配置：
  - `min-replicas-to-write N`：至少 N 个从库连接正常才接受写；
  - `min-replicas-max-lag T`：从库复制延迟不超过 T 秒。
- 本质是 Redis 主从异步复制，**无法完全避免数据丢失**，需要强一致应使用 etcd/ZooKeeper 等共识系统。

五、缓存问题

17. 缓存穿透与布隆过滤器

- 定义：请求的数据**在缓存和数据库中都不存在**，每次都打到数据库（常见于恶意攻击）。
- 方案：
  - **参数校验**：拦截非法 ID；
  - **缓存空值**：数据库查不到也缓存空对象，设置**较短 TTL**；
  - **布隆过滤器**：先判断 key 是否可能存在，不存在直接返回。

延伸 / 易错点

- **布隆过滤器原理**：bit 数组 + k 个哈希函数，插入时置 k 位为 1；查询时任一位为 0 则**一定不存在**，全为 1 则**可能存在**（有误判）。**不支持删除**（可用计数布隆过滤器/布谷鸟过滤器）。
- **误判率**：与 bit 数组大小 m、哈希函数个数 k、元素数 n 有关；m 越大误判越低。

18. 缓存击穿

- 定义：某个**热点 key 过期**瞬间，大量并发请求同时打到数据库。
- 方案：
  - **互斥锁/单飞**：只允许一个请求查库回填，其余等待或重试（Go 中可用 `golang.org/x/sync/singleflight`）；
  - **逻辑过期**：缓存不设物理 TTL，value 中存过期时间，发现过期后异步重建，期间返回旧值；
  - **热点 key 不过期** / 提前续期。

延伸 / 易错点

- **取舍**：互斥锁保证一致性但有等待；逻辑过期可用性高但短时间返回旧数据。

19. 缓存雪崩

- 定义：**大量 key 同时过期**或 **Redis 宕机**，请求集中打到数据库。
- 方案：
  - 过期时间加**随机值**打散；
  - **高可用**：主从 + 哨兵 / Cluster；
  - **多级缓存**：本地缓存（如 Go 的 freecache/bigcache、Java 的 Caffeine）+ Redis；
  - **限流、熔断、降级**保护数据库；
  - 缓存预热。

延伸 / 易错点

- **三者区分**：穿透 = 数据不存在；击穿 = 单个热点 key 失效；雪崩 = 大面积失效或服务宕机。

20. 大 key 与热 key

- **大 key**：value 过大（如 String > 10KB、集合元素数万以上，具体阈值依业务）。危害：阻塞主线程、网络拥塞、内存不均、删除时阻塞。
  - 发现：`redis-cli --bigkeys`、`MEMORY USAGE key`、离线分析 RDB。
  - 处理：**拆分**（按字段/分片拆成多个 key）、压缩、使用 `UNLINK` **异步删除**（4.0+），开启 `lazyfree-lazy-*`。
- **热 key**：访问频率极高，单节点压力过大。
  - 发现：`redis-cli --hotkeys`（需 LFU 策略）、客户端/代理层统计。

- 处理：**本地缓存**、**热 key 复制多份**（加后缀分散到不同节点）、读写分离。

21. KEYS 与 SCAN

- **KEYS pattern**：一次遍历全部 key，O(N)，**会阻塞主线程**，生产环境禁用（可 `rename-command` 屏蔽）。
- **SCAN cursor [MATCH] [COUNT]**：基于游标**分批增量迭代**，每次只返回一小批，不长时间阻塞。
  - COUNT 只是**提示**，每次返回数量不固定；游标返回 0 表示遍历结束。
  - **保证**：完整遍历期间**一直存在**的 key 一定会被返回；**可能返回重复 key**，需客户端去重；遍历中新增/删除的 key 可能出现也可能不出现。
  - 采用**高位进位（反向二进制）**的游标顺序，使扩容/缩容 rehash 时也不会漏掉已存在的 key。
- 集合类型对应有 `HSCAN`、`SSCAN`、`ZSCAN`。

22. Redis 变慢排查思路

- **慢命令**：`SLOWLOG GET` 查看慢日志（阈值 `slowlog-log-slower-than`）；排查 O(N) 命令、大 key 操作。
- **大 key**：读写和删除大 key 耗时长 → 拆分、`UNLINK`。
- **集中过期**：大量 key 同一时刻过期，主动过期清理占用主线程 → 过期时间加随机值。
- **内存达到上限**：每次写入前都要淘汰 key → 调整 maxmemory、淘汰策略，拆分实例。
- **fork 耗时**：`INFO` 中 `latest_fork_usec`；控制实例内存大小，避开高峰做持久化。
- **AOF 刷盘**：`appendfsync always` 或磁盘 IO 压力大 → everysec，`no-appendfsync-on-rewrite yes`。
- **操作系统**：内存 swap、透明大页（THP，COW 时复制 2MB 大页）、CPU 绑核不当、网络带宽打满。
- 工具：`redis-cli --latency`、`LATENCY DOCTOR`（需开启 latency-monitor）。

六、分布式锁

23. Redis 分布式锁

- 加锁：`SET lock_key <唯一值uuid> NX PX 30000` —— **NX 保证互斥，PX 设置过期防止死锁，二者一条命令保证原子性**。
- 解锁：用 **Lua 脚本**先比较 value 是否为自己的 uuid 再 `DEL`，保证「判断+删除」原子，**防止误删他人的锁**。

```
        if redis.call("get", KEYS[1]) == ARGV[1] then
          return redis.call("del", KEYS[1])
```

```
    else
      return 0
    end
```

- 常见坑：
  - **锁过期但业务未完成** → 自动续期（看门狗）；
  - **不可重入** → 用 Hash 结构记录持有者和重入次数；
  - **主从切换锁丢失**：主库加锁后未同步到从库即宕机，新主上没有锁 → Redlock 或改用强一致组件。

延伸 / 易错点

- **Redisson 看门狗**：未指定 leaseTime 时，默认锁超时 **30 秒**，后台每 **10 秒（超时的 1/3）** 续期；客户端宕机则不再续期，锁自然过期。
- **Redlock**：向 **N（通常 5）个相互独立的 Redis 主节点**依次加锁，**超过半数（≥ N/2+1）成功且总耗时小于锁有效期**才算成功，失败则向所有节点释放。
- **Redlock 争议**：Martin Kleppmann 指出其依赖时钟假设，GC 停顿/时钟跳变下仍可能不安全，建议使用 **fencing token（单调递增令牌）** 由存储端校验。

24. Redis / ZooKeeper / etcd 锁对比

- **Redis**：性能高，基于过期时间，AP 倾向，极端情况可能失效。
- **ZooKeeper**：**临时顺序节点** + Watch 前一个节点，会话断开节点自动删除；CP，可靠性高但性能较低。
- **etcd**：基于 **Raft** 强一致，使用 **Lease（租约）+ Revision**，按 revision 最小者获得锁，Watch 前一个 key；Go 生态常用（`clientv3/concurrency`）。
- 选择：允许极少数失效、追求性能 → Redis；要求正确性 → etcd/ZooKeeper。

七、缓存一致性

25. Cache Aside 与缓存一致性

- 最常用 **Cache Aside 旁路缓存**：
  - 读：先读缓存，命中返回；未命中读数据库，再写入缓存；
  - 写：**先更新数据库，再删除缓存**。
- **为什么删除缓存而不是更新缓存？** —— 并发写时更新缓存的顺序可能与数据库不一致；且缓存值可能需复杂计算、写多读少时更新是浪费。删除后由下次读懒加载。
- **为什么先更新库再删缓存，而不是先删缓存？** —— 先删缓存时，另一读请求在库更新前读到旧值并回填缓存，导致长期脏数据。先更新库的异常窗口要求「读请求在缓存失效时读到旧值，并在写请求删缓存**之后**才回填」，由于缓存写入远快于数据库写入，概率很低。

- **删除缓存失败怎么办？** —— **重试**：将删除操作投递到**消息队列**重试；或**订阅 binlog**（Canal 等）异步删除缓存，与业务解耦。
- 兜底：缓存**设置过期时间**，保证最终一致。

延伸 / 易错点

- **延迟双删**：先删缓存 → 更新数据库 → **sleep 一段时间（大于一次读请求耗时 + 主从延迟）再删一次**，用于「先删缓存」方案或读写分离从库延迟场景；缺点是延迟时间难确定。
- **强一致**：缓存 + 数据库很难强一致，需分布式读写锁等，牺牲性能；一般追求**最终一致性**。

26. 其他缓存读写模式

- **Read/Write Through**：应用只和缓存交互，由缓存层同步读写数据库（需缓存组件支持）。
- **Write Behind（Write Back）**：只写缓存，**异步批量**写回数据库；性能高，但宕机可能丢数据，一致性弱（类似 CPU Cache 写回、MySQL Buffer Pool）。

八、其他常考

27. 事务、Lua 脚本与 Pipeline

- **事务**：`MULTI` 开启 → 命令入队 → `EXEC` 执行；`WATCH` 实现乐观锁（被监视 key 变化则 EXEC 返回空）。
  - 保证**顺序执行、不被打断**（隔离）；**不支持回滚**：运行时某条出错，其他命令照常执行；入队时语法错误则整个事务不执行。
- **Lua 脚本**：`EVAL`，整个脚本**原子执行**，可包含逻辑判断；脚本不宜过长，否则阻塞。7.0 引入 Function。
- **Pipeline**：客户端将多条命令**批量发送**，减少 RTT；**不保证原子性**，仅是网络优化。

延伸 / 易错点

- **与 ACID 的关系**：保证隔离性，原子性不完整（无回滚），持久性取决于持久化配置。

28. 典型应用：延迟队列、排行榜、限流

- **延迟队列**：ZSet，score = 执行时间戳，轮询 `ZRANGEBYSCORE key 0 now` 取到期任务，用 `ZREM` 成功与否保证只被一个消费者处理（或用 Lua 原子取出）。
- **排行榜**：ZSet `ZINCRBY` 加分，`ZREVRANGE key 0 9 WITHSCORES` 取 Top10，`ZREVRANK` 查排名。
- **限流**：
  - 固定窗口：`INCR` + `EXPIRE`；
  - 滑动窗口：ZSet 存请求时间戳，删除窗口外记录后计数；
  - 令牌桶：Lua 脚本维护令牌数与上次补充时间。

29. 秒杀场景与防超卖

- **流量分层拦截**：前端静态化 + CDN、按钮置灰防重复提交；网关**限流**；后端用 Redis 承接高并发读写，数据库只处理最终订单。
- **库存预扣（Redis）**：活动前把库存加载到 Redis，用 **Lua 脚本原子执行「判断库存 > 0 → 扣减 → 记录用户已购」**，避免并发超卖与重复购买。
- **异步下单**：扣减成功后发送消息到 **MQ**，消费者异步创建订单、扣减数据库库存，削峰填谷。
- **数据库兜底**：`UPDATE stock SET count = count - 1 WHERE id = ? AND count > 0`（条件更新，影响行数为 0 即失败）；**一人一单**用（用户 ID, 商品 ID）**唯一索引**保证。
- **一致性**：消息消费要**幂等**；下单失败/超时未支付需回补库存；Redis 与数据库库存定期对账。